

EECS 348 - Software Engineering I - Assignment 3

GenAI Code Analysis

Charvi Reddy Konudula

a. GenAI Programs

Two GenAI tools were used to generate an initial C++ implementation: **ChatGPT** (OpenAI) and **Google Gemini**.

b. How Each Was Accessed

Both tools were accessed through their web interfaces in the Google Chrome browser: ChatGPT at chat.openai.com and Gemini at gemini.google.com. No desktop apps, browser extensions or API access were used; both were plain browser chat sessions.

c. Prompt Used (identical for both GenAIs)

Write C++ code using classes/objects (not standalone functions) for a program that prioritizes emails for a busy CEO.

Requirements:

- Implement a MaxHeap from scratch as a priority queue using a list/array-based implementation (e.g., std::vector or dynamic array as the backing store). Do NOT use std::priority_queue, std::make_heap, or any pre-existing heap module.
- Read commands from a text file (path given as argv[1]). Commands:
 - "EMAIL <sender category>,<subject>,<date>" (space after EMAIL, then comma-delimited). Sender category is one of Boss, Subordinate, Peer, ImportantPerson, OtherPerson. Subject may contain spaces but no commas. Date is MM-DD-YYYY.
 - "NEXT": display the highest-priority email as:
Sender: <category>
Subject: <subject>
Date: <date>
preceded by the line "Next email:"
 - "READ": remove the highest-priority email from the queue (no output).
 - "COUNT": print "There are N emails to read."
- Priority: Boss > Subordinate > Peer > ImportantPerson > OtherPerson. Within the same category, the NEWEST email (latest date) comes first.
- Edge cases: NEXT or READ on an empty queue must not crash; two NEXTs in a row print the same email; two READs in a row delete two emails without displaying.
- Must compile with g++ on a Linux server.

Sample input:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Expected output:

```
There are 6 emails to read.
Next email:
```

Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.

d. Generated Code (Raw, Unedited Output)

d.1 - ChatGPT

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

// -----
// Email class
// -----
class Email {
private:
    string sender;
    string subject;
    string date;
    int priority;
    int dateValue;

public:
    Email(string s, string sub, string d) {
        sender = s;
        subject = sub;
        date = d;

        // Assign priority
        if (sender == "Boss")
            priority = 5;
        else if (sender == "Subordinate")
            priority = 4;
        else if (sender == "Peer")
            priority = 3;
        else if (sender == "ImportantPerson")
            priority = 2;
        else
            priority = 1;

        // Convert MM-DD-YYYY into a number that can be compared.
        // YYYYMMDD format makes newer dates have larger values.
        int month = stoi(date.substr(0, 2));
        int day = stoi(date.substr(3, 2));
        int year = stoi(date.substr(6, 4));

        dateValue = year * 10000 + month * 100 + day;
    }

    string getSender() const {
        return sender;
    }

    string getSubject() const {
        return subject;
    }

    string getDate() const {
        return date;
    }
}
```

```

// Returns true if this email has higher priority than other.
bool higherPriorityThan(const Email& other) const {
    if (priority != other.priority)
        return priority > other.priority;

    // Same sender category: newest email comes first.
    if (dateValue != other.dateValue)
        return dateValue > other.dateValue;

    // If both category and date are the same,
    // keep the existing order.
    return false;
}
};

// -----
// MaxHeap class
// -----
class MaxHeap {
private:
    vector<Email> heap;

    int parent(int index) {
        return (index - 1) / 2;
    }

    int leftChild(int index) {
        return 2 * index + 1;
    }

    int rightChild(int index) {
        return 2 * index + 2;
    }

    void swapEmails(int first, int second) {
        Email temp = heap[first];
        heap[first] = heap[second];
        heap[second] = temp;
    }

    // Move an item upward until the heap property is restored.
    void heapifyUp(int index) {
        while (index > 0) {
            int p = parent(index);

            if (heap[index].higherPriorityThan(heap[p])) {
                swapEmails(index, p);
                index = p;
            }
            else {
                break;
            }
        }
    }

    // Move an item downward until the heap property is restored.
    void heapifyDown(int index) {
        int size = heap.size();

        while (true) {
            int left = leftChild(index);
            int right = rightChild(index);
            int highest = index;

            if (left < size &&
                heap[left].higherPriorityThan(heap[highest])) {
                highest = left;
            }

            if (right < size &&
                heap[right].higherPriorityThan(heap[highest])) {
                highest = right;
            }

            if (highest != index) {

```

```

        swapEmails(index, highest);
        index = highest;
    }
    else {
        break;
    }
}

public:
    // Add an email to the heap.
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    // Check whether the heap is empty.
    bool isEmpty() const {
        return heap.empty();
    }

    // Return the highest-priority email without removing it.
    Email* peek() {
        if (heap.empty())
            return nullptr;

        return &heap[0];
    }

    // Remove the highest-priority email.
    void removeMax() {
        if (heap.empty())
            return;

        // Move the last element to the root.
        heap[0] = heap.back();
        heap.pop_back();

        if (!heap.empty())
            heapifyDown(0);
    }

    int size() const {
        return heap.size();
    }
};

// -----
// Main program
// -----
int main(int argc, char* argv[]) {

    // Make sure a file path was provided.
    if (argc < 2) {
        return 1;
    }

    ifstream inputFile(argv[1]);

    if (!inputFile.is_open()) {
        return 1;
    }

    MaxHeap emailQueue;

    string line;

    while (getline(inputFile, line)) {

        if (line.empty())
            continue;

        // -----
        // EMAIL command

```

```

// -----
if (line.substr(0, 6) == "EMAIL") {

    // Remove "EMAIL " from the beginning.
    string data = line.substr(6);

    // Remove the space after EMAIL.
    if (!data.empty() && data[0] == ' ')
        data = data.substr(1);

    // Find the commas.
    size_t firstComma = data.find(',');
    size_t secondComma = data.find(',', firstComma + 1);

    string sender = data.substr(0, firstComma);

    string subject = data.substr(
        firstComma + 1,
        secondComma - firstComma - 1
    );

    string date = data.substr(secondComma + 1);

    Email email(sender, subject, date);

    emailQueue.insert(email);
}

// -----
// NEXT command
// -----
else if (line == "NEXT") {

    Email* nextEmail = emailQueue.peek();

    // Empty queue: do nothing.
    if (nextEmail != nullptr) {
        cout << "Next email:" << endl;
        cout << "Sender: " << nextEmail->getSender() << endl;
        cout << "Subject: " << nextEmail->getSubject() << endl;
        cout << "Date: " << nextEmail->getDate() << endl;
    }
}

// -----
// READ command
// -----
else if (line == "READ") {

    // removeMax() safely does nothing if empty.
    emailQueue.removeMax();
}

// -----
// COUNT command
// -----
else if (line == "COUNT") {

    cout << "There are "
         << emailQueue.size()
         << " emails to read."
         << endl;
}
}

inputFile.close();

return 0;
}

```

d.2 - Gemini

```
#include <iostream>
```

```

#include <fstream>
#include <string>
#include <vector>
#include <sstream>

class Email {
private:
    std::string category;
    std::string subject;
    std::string dateStr;
    int priorityVal;
    long dateVal;

    int getCategoryPriority(const std::string& cat) const {
        if (cat == "Boss") return 5;
        if (cat == "Subordinate") return 4;
        if (cat == "Peer") return 3;
        if (cat == "ImportantPerson") return 2;
        if (cat == "OtherPerson") return 1;
        return 0;
    }

    long parseDate(const std::string& d) const {
        // Expected format: MM-DD-YYYY
        int mm = std::stoi(d.substr(0, 2));
        int dd = std::stoi(d.substr(3, 2));
        int yyyy = std::stoi(d.substr(6, 4));
        return yyyy * 10000L + mm * 100L + dd;
    }

public:
    Email(const std::string& cat, const std::string& subj, const std::string& date)
        : category(cat), subject(subj), dateStr(date) {
        priorityVal = getCategoryPriority(cat);
        dateVal = parseDate(date);
    }

    std::string getCategory() const { return category; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return dateStr; }

    bool operator<(const Email& other) const {
        if (priorityVal != other.priorityVal) {
            return priorityVal < other.priorityVal;
        }
        return dateVal < other.dateVal; // Newest/latest date comes first in max-heap
    }
};

class MaxHeap {
private:
    std::vector<Email> heap;

    void siftUp(int idx) {
        while (idx > 0) {
            int parent = (idx - 1) / 2;
            if (heap[parent] < heap[idx]) {
                std::swap(heap[parent], heap[idx]);
                idx = parent;
            } else {
                break;
            }
        }
    }

    void siftDown(int idx) {
        int size = heap.size();
        while (true) {
            int left = 2 * idx + 1;
            int right = 2 * idx + 2;
            int largest = idx;

            if (left < size && heap[largest] < heap[left]) {
                largest = left;
            }
        }
    }
};

```

```

        if (right < size && heap[largest] < heap[right]) {
            largest = right;
        }
        if (largest != idx) {
            std::swap(heap[idx], heap[largest]);
            idx = largest;
        } else {
            break;
        }
    }
}

public:
    void insert(const Email& email) {
        heap.push_back(email);
        siftUp(heap.size() - 1);
    }

    bool isEmpty() const {
        return heap.empty();
    }

    int size() const {
        return heap.size();
    }

    const Email& getMax() const {
        return heap[0];
    }

    void removeMax() {
        if (heap.empty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            siftDown(0);
        }
    }
};

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: " << argv[0] << " <input_file>\n";
        return 1;
    }

    std::ifstream file(argv[1]);
    if (!file.is_open()) {
        std::cerr << "Error opening file: " << argv[1] << "\n";
        return 1;
    }

    MaxHeap emailHeap;
    std::string line;

    while (std::getline(file, line)) {
        if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            std::string data = line.substr(6);
            size_t firstComma = data.find(',');
            size_t secondComma = data.find(',', firstComma + 1);

            std::string cat = data.substr(0, firstComma);
            std::string subj = data.substr(firstComma + 1, secondComma - firstComma - 1);
            std::string date = data.substr(secondComma + 1);

            emailHeap.insert(Email(cat, subj, date));
        }
        else if (line == "NEXT") {
            if (!emailHeap.isEmpty()) {
                const Email& top = emailHeap.getMax();
                std::cout << "Next email:\n";
                std::cout << "Sender: " << top.getCategory() << "\n";
                std::cout << "Subject: " << top.getSubject() << "\n";
            }
        }
    }
}

```

```
        std::cout << "Date: " << top.getDate() << "\n";
    }
}
else if (line == "READ") {
    emailHeap.removeMax();
}
else if (line == "COUNT") {
    std::cout << "There are " << emailHeap.size() << " emails to read.\n";
}
}
return 0;
}
```


e. Analysis of the Two Programs

Method

Both programs were compiled with `g++ -std=c++17` and run on (1) the assignment sample file, (2) an edge-case file (empty NEXT/READ, double NEXT, double READ, malformed lines), (3) a file with Windows (CRLF) line endings, and (4) a tie test with four Peer emails (A, B, C, D) that all have the same date. Execution time and space are analyzed from operation counts and data structures rather than from a benchmark.

Measured results

- **Sample file:** Gemini prints exactly the expected output. ChatGPT prints "There are 0 emails to read." twice and no NEXT output, because no email is ever inserted.
- **CRLF file:** in both programs the commands COUNT, NEXT and READ are never recognized, because the lines are "COUNT\r", "NEXT\r" and "READ\r". EMAIL lines are still accepted. After converting the file to LF, Gemini is correct and ChatGPT still fails because of its EMAIL bug.
- **Malformed line "EMAIL Peer,x" (missing comma):** Gemini crashes with an uncaught `std::invalid_argument` from `stoi` (exit code 134). ChatGPT, with only its EMAIL bug patched, crashes the same way.
- **Malformed date "EMAIL Peer,x,1-1-2025":** Gemini does not crash. It silently accepts the email and computes a meaningless date value, so the error goes unnoticed.
- **Ties (four Peer emails A, B, C, D, same date):** both programs read them in the order A, D, C, B. For ChatGPT this is after patching only the EMAIL bug.

Correctness

Gemini. The core logic is right. `rfind("EMAIL ", 0) == 0` recognizes EMAIL lines, and dates become YYYYMMDD integers, so newest-first works across years. Weaknesses:

- CRLF line endings make COUNT, NEXT and READ fail silently.
- There is no input validation: a missing comma crashes the program and a malformed date is accepted with a wrong value.
- Unknown sender categories are accepted with priority 0.
- Ties come out in arbitrary order, because a binary heap is not stable.
- `getMax()` reads `heap[0]` without checking for an empty heap (callers must check `isEmpty()` first).

ChatGPT. The heap and priority logic are sound by inspection, but `line.substr(0, 6) == "EMAIL"` compares a 6-character string ("EMAIL " with a trailing space) to a 5-character one, so it is never true. No email is inserted and the sample test fails. Other issues:

- No CRLF handling and no validation (same crash on a missing comma).
- Unknown categories silently get priority 1.
- The comment "keep the existing order" for ties is misleading, since a heap does not preserve arrival order, so ties come out in arbitrary order.
- A bad or missing input file returns 1 with no message.

Execution time

Both are asymptotically optimal. `insert` and `removeMax` are $O(\log n)$, `peek/getMax` and `size` are $O(1)$, so processing n emails costs $O(n \log n)$. Dates are parsed once in the constructor, so each comparison is $O(1)$. The difference is the constant factor. ChatGPT's `swapEmails` makes three full copies of an Email (each holding three strings) per swap, while Gemini uses `std::swap`, which moves the strings. Both copy `heap.back()` in

removeMax instead of moving it.

Space complexity

Both use $O(n)$ space for the `std::vector`. Both store the date string and its integer value, which is slightly redundant. Gemini's long `dateVal` is larger than `YYYYMMDD` needs. ChatGPT's `swap` allocates temporary copies of three strings on every swap, but they are freed immediately, so peak space is essentially the same.

Maintainability

Gemini. Clean Email/MaxHeap split, small private helpers, const-correct getters, `std::` prefixes and const Email& parameters. Weaknesses: magic numbers 1-5, command parsing inline in main, very few comments, no prologue, an unused `<sstream>` include, and implicit `size_t`-to-int conversions.

ChatGPT. Good section comments and descriptive names such as `higherPriorityThan` and `heapifyUp`.

Weaknesses:

- It uses "using namespace std;".
- It uses magic numbers in an if/else chain.
- `peek()` returns a raw pointer that dangles after any insert or remove.
- `parent/leftChild/rightChild` are not const.
- Parsing is inline in main.
- It has an unused `<sstream>` include and no prologue.

f. Selection and Justification

I selected **Gemini's code** as the baseline.

The deciding evidence is the test result. Gemini's program produces the exact expected output on the assignment's sample file, while ChatGPT's prints "There are 0 emails to read." twice and never shows a NEXT email, because of the `substr(0, 6) == "EMAIL"` bug. Passing the sample is the first requirement of the assignment, so ChatGPT's version would need a fix before it could even be graded. The two are otherwise close in design, and the secondary reasons also favor Gemini:

- It uses `std::swap`, which moves strings instead of making three copies per swap.
- It has no dangling-pointer accessor.
- It avoids "using namespace std".
- Its API is const-correct.

The trade-off I accepted is that ChatGPT has slightly better section comments. Comments were going to be added everywhere anyway, and Gemini's weaknesses (CRLF, validation, ties) can be fixed in a focused way.

g. Improvements Made to Gemini's Code

Every block in `main.cpp` is labeled `[GEMINI]` (kept as generated), `[GEMINI-MODIFIED]` (Gemini's code, modified for this assignment) or `[ADDED]` (new code). I reviewed, compiled, tested and integrated every change myself on the EECS Cycle server.

Correctness

- **CRLF:** a `trim()` helper strips spaces, tabs and `\r` from every line and field. The CRLF file now processes correctly.
- **Crashes and bad dates:** `stoi/substr` were replaced by a validated manual date parse, and comma checks were added. Command handling is wrapped in try/catch, which prints a note to `stderr` and skips the line. A

bad line no longer crashes the program, and the exit code stays 0. A date such as 1-1-2025 is now rejected instead of silently accepted.

- **Unknown category:** `parseCategory()` throws on unknown names, so "EMAIL Bad,x,01-01-2025" is rejected.
- **Ties:** an arrival sequence number is the final tie-breaker in `operator<`. The tie test now reads A, B, C, D, which is deterministic first-come first-served.
- **Empty heap:** `getMax()` throws `std::out_of_range` when empty, and callers check `isEmpty()` first. NEXT and READ on an empty inbox print nothing and do not crash.

Execution time

`insert(Email)` takes its argument by value and uses `std::move`, and `removeMax` moves the last element instead of copying it. Both operations are still $O(\log n)$ but make fewer string copies. The validation and tie-breaker add a small constant cost per comparison, so I do not claim a measured speedup over Gemini's original.

Space complexity

`std::size_t` is used for indexes and sizes, the getters return `const std::string&`, and the unused `<sstream>` include was removed. Space is still $O(n)$. The 8-byte sequence field adds a small constant per email, a cost I accepted for deterministic ordering.

Maintainability

The magic numbers were replaced by enum class `SenderCategory`. Command handling moved out of main into a `CommandProcessor` class (`handleEmail`, `handleNext`, `handleRead`, `handleCount`, `run`), so main only opens the file. A prologue and a comment on every line were added, and every block is labeled with its source. The result is named priorities, single-responsibility classes (`Email`, `MaxHeap`, `CommandProcessor`), full comments and clear attribution.

Note on time and space

The improvements mainly raise correctness and maintainability. The asymptotic costs are unchanged ($O(\log n)$ per heap operation, $O(n)$ space).

Final Verification

The final program was compiled on the EECS Cycle server (`cycle1.eecs.ku.edu`) with `g++ -std=c++17 -o email_priority main.cpp`. The executable pushed to GitHub is that Linux x86-64 build.

Sample file output on the Cycle server:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

Edge-case file on the Cycle server (empty NEXT/READ, double NEXT, double READ, bad lines):

```
There are 0 emails to read.
Next email:
Sender: Peer
Subject: A
```

Date: 01-01-2025
Next email:
Sender: Peer
Subject: A
Date: 01-01-2025
Next email:
Sender: Peer
Subject: C
Date: 12-31-2024
There are 0 emails to read.

stderr:
Skipping bad line (unknown sender category): EMAIL Bad,x,01-01-2025
Skipping bad line (missing second comma): EMAIL Peer,x
Skipping bad line (date must be MM-DD-YYYY): EMAIL Peer,x,1-1-2025

The "Skipping bad line" messages go to stderr for my own testing. A test file with only valid commands will not print them.